

Universidad Rafael Landívar
Facultad de Ingeniería
Licenciatura en Ingeniería en Informática y Sistemas
Manejo e Implementación de Archivos Sección: 2
Docente: Ing. Luis Rojas

SISTEMA DE GESTIÓN DE ESTUDIANTES

Estudiantes: Marco Donadio, Sebastián
Villeda, Martín Morales, Gerber Pérez, Edwin
Santisteban

Carné: 1076925, 1032625, 121705, 1165825, 1047425

Guatemala 30 de agosto de 2026

Contenido

Sección	Tema
1	Problema y objetivos
2	Requisitos e instalación
3	Ingreso y registro
4	Búsqueda y listado
5	Modificación de estudiantes
6	Eliminación y contraseña
7	Arquitectura del sistema
8	Estructura del XML
9	Persistencia y cifrado
10	Validaciones y solución de problemas
11	Control de versiones
12	Conclusiones y referencias

Lectura del manual

Las opciones se identifican con el número que aparece en la consola. ENTER confirma una entrada o continúa después de un resultado. Las capturas muestran ejecuciones reales del sistema y conservan los valores utilizados en esas pruebas.

El documento Evidencia de funcionamiento reúne las pruebas y sus resultados visibles. La numeración original de las capturas corresponde al archivo Capturas.docx.

1. Problema y objetivos

Descripción del problema

La gestión de estudiantes requiere conservar datos identificables, localizar registros y actualizar información sin perder los cambios al cerrar la aplicación. Un archivo desorganizado o sin validaciones facilita la duplicación de registros y dificulta la recuperación de información.

El sistema resuelve estas operaciones mediante una aplicación de consola en C#. Utiliza una estructura XML para representar a los estudiantes y protege el contenido almacenado mediante cifrado. No requiere un servidor de base de datos.

Objetivo general

Administrar información de estudiantes mediante archivos XML, con operaciones de registro, consulta, actualización y eliminación, validación de datos y almacenamiento cifrado.

Objetivos específicos

Conservar los registros entre ejecuciones; buscar por un carné único; validar la información antes de guardarla; organizar las responsabilidades en clases; y proteger el archivo mediante una contraseña de acceso.

Opción	Funcionalidad	Resultado
1	Registrar	Agrega un estudiante válido.
2	Buscar por carné	Muestra un registro o informa que no existe.
3	Modificar	Actualiza un campo del estudiante.
4	Eliminar	Quita un registro después de confirmar.
5	Listar	Muestra los estudiantes almacenados.
6	Cambiar contraseña	Vuelve a cifrar con la nueva contraseña.
0	Salir	Finaliza la sesión.

El alcance corresponde a una aplicación local. No incluye cuentas individuales, permisos por rol, recuperación automática de contraseñas ni acceso simultáneo coordinado entre varios usuarios.

2. Requisitos e instalación

Para compilar desde el código fuente se necesita el SDK de .NET 8 y una terminal. El proyecto declara net8.0. Para ejecutar una compilación dependiente del framework se necesita un runtime compatible de .NET 8; instalar únicamente un runtime de otra versión principal no garantiza el arranque.

En Windows puede utilizarse Visual Studio Code, Visual Studio o PowerShell. El editor es opcional. Se requieren permisos de lectura y escritura en la carpeta que almacenará los datos.

Ejecución desde el proyecto

Abra una terminal en la carpeta que contiene Proyecto_1.csproj. Compruebe .NET, compile y ejecute:

```
dotnet --list-sdks dotnet --list-runtimes
dotnet build Proyecto_1.csproj dotnet
run --project Proyecto_1.csproj
```

Ubicación de los archivos

Forma de inicio	Carpeta de datos
IniciarProyecto.ps1	C:\GestionEstudiantes\Datos
Ejecución directa sin variable configurada	%LOCALAPPDATA%\GestionEstudiantes\Datos
Variable GESTION_ESTUDIANTES_DATOS definida	La ruta indicada por esa variable.

El archivo activo se llama prueba_XML.xml. La consola indica la carpeta utilizada al iniciar. Es necesario conservar la misma ubicación para volver a abrir los mismos registros. La carpeta Datos incluida en el código fuente no se selecciona automáticamente en esta configuración. **Inicio mediante el script**

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\IniciarProyecto.ps1
```

El permiso de ejecución de este comando se limita al proceso iniciado. Si la carpeta de C: no permite escritura, puede ejecutar el programa directamente para utilizar la carpeta personal o configurar una ruta con permisos.

Ejecución desde PowerShell

El sistema puede ejecutarse desde PowerShell. Abra una terminal en la carpeta que contiene Proyecto_1.csproj y escriba:

```
dotnet run --project Proyecto_1.csproj
```

También puede utilizar el script de inicio, que configura la carpeta de datos y ejecuta el proyecto:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\IniciarProyecto.ps1
```

Ejecución mediante el archivo ejecutable

El sistema también puede iniciarse haciendo doble clic en Proyecto_1.exe, sin abrir un editor de código. Otra opción es abrir PowerShell en la carpeta del ejecutable y escribir:

```
.\Proyecto_1.exe
```

El ejecutable debe conservarse junto con los archivos que acompañan su distribución. Si la aplicación se publicó como autocontenida, incluye el entorno necesario para funcionar. Si depende del framework, requiere el runtime de .NET 8 instalado.

Al abrir el ejecutable directamente, el programa utiliza la carpeta %LOCALAPPDATA%\GestionEstudiantes\Datos, salvo que esté definida la variable GESTION_ESTUDIANTES_DATOS. La ubicación activa se muestra en la consola al iniciar.

3. Ingreso y registro

Primera ejecución y acceso

Si el archivo no existe, el sistema solicita una contraseña nueva y su confirmación. Crea la estructura vacía y la guarda cifrada. En las siguientes ejecuciones solicita la contraseña del archivo. La escritura de la contraseña no se muestra en pantalla. ENTER en una solicitud de contraseña permite cancelar.

```
La carpeta de datos ya existe:  
C:\Users\gerbe\AppData\Local\GestionEstudiantes\Datos  
  
Primera ejecución: se creará un archivo sin estudiantes.  
Cree una contraseña para proteger sus datos.  
Nueva contraseña (ENTER para cancelar):  
Confirme la nueva contraseña:  
  
Datos descifrados en memoria correctamente.  
El archivo permanece cifrado en la carpeta de datos.  
Presione ENTER para abrir el menú principal.
```

Figura 1. Creación de contraseña y preparación del archivo.

Registro de estudiantes

Seleccione 1. Ingrese carné, nombres, apellidos, carrera y correo. El menú solicita un carné de seis dígitos y comprueba que no esté registrado; también solicita un correo con formato válido y sin coincidencia con otro registro. Todos los campos son obligatorios.

Cuando aparece “Estudiante registrado exitosamente”, el registro se ha guardado. Presione ENTER para regresar al menú. Si una validación falla, atienda el mensaje y corrija los datos solicitados.

```
=== REGISTRO DE ESTUDIANTE ===  
  
Carné (entero de 6 dígitos): 116582  
Nombre(s): Gerber Mariano  
Apellidos: Perez Gomez  
Carrera: Ing. Sistemas  
Correo: gmperezgom@correo.url.edu.gt  
  
Estudiante registrado exitosamente.  
  
Presione ENTER para continuar...
```

Figura 2. Registro del estudiante con carné 116582.

4. Búsqueda y listado

Buscar un estudiante

Seleccione 2 e ingrese el carné completo. El sistema lee los datos, busca el identificador y muestra carné, nombre completo, carrera y correo. Si no encuentra una coincidencia, informa que el estudiante no existe.

```
=== BÚSQUEDA DE ESTUDIANTE ===  
  
Ingrese el carné del estudiante: 116582  
  
Estudiante encontrado:  
Carné: 116582  
Nombre: Gerber Mariano Perez Gomez  
Carrera: Ing. Sistemas  
Correo: gmperezgom@correo.url.edu.gt  
  
Presione ENTER para continuar...
```

Figura 3. Recuperación del estudiante 116582 mediante su carné.

Consultar todos los estudiantes

Seleccione 5. La consola muestra el total y los datos de cada estudiante. Si el archivo válido no contiene registros, indica que no existen estudiantes registrados.

```
=== LISTADO DE ESTUDIANTES ===  
  
Total de estudiantes: 1  
  
Carné: 116325  
Nombre: Luis Perez  
Carrera: Ing Civil  
Correo: lperez@correo.url.edu.gt  
-----  
  
Presione ENTER para continuar...
```

Figura 4. Listado con un estudiante almacenado.

La búsqueda se realiza sobre la lista reconstruida desde el archivo. La implementación utiliza recorrido secuencial; no mantiene un índice persistente ni una base de datos relacional.

5. Modificación de estudiantes

Seleccione 3 e ingrese el carné. Cuando el registro aparece, elija el campo que desea cambiar: 1 para nombres, 2 para apellidos, 3 para carrera o 4 para correo. Escriba el nuevo valor y confirme con ENTER.

El carné se conserva como identificador. El programa prepara una copia con el cambio, valida los datos y la guarda. Después de un cambio exitoso muestra el registro actualizado y permite editar otro campo. Seleccione 0 para volver al menú principal.

```
Datos actuales:
Carné: 116582
Nombre: Mario Gonzalez
Carrera: Ing. Sistemas
Correo: gmperezgom@correo.url.edu.gt

1. Nombre(s)
2. Apellidos
3. Carrera
4. Correo
0. Volver al menú principal
Seleccione el campo: 3
Carrera (ENTER para cancelar): Ing. Industrial

Estudiante modificado correctamente.

Carné: 116582
Nombre: Mario Gonzalez
Carrera: Ing. Industrial
Correo: gmperezgom@correo.url.edu.gt

1. Nombre(s)
2. Apellidos
3. Carrera
4. Correo
0. Volver al menú principal
Seleccione el campo: |
```

Figura 5. Cambio de carrera de Ing. Sistemas a Ing. Industrial. El carné 116582 permanece igual.

Si el carné no existe, puede intentar con otro. ENTER al solicitar el carné cancela la operación. ENTER al solicitar un nuevo valor cancela ese cambio. El menú también comprueba que un correo nuevo no coincida con el de otro estudiante.

6. Eliminación y contraseña

Eliminar un registro

Seleccione 4 e ingrese el carné. Revise los datos que aparecen. Responda S para confirmar la eliminación; cualquier otra respuesta cancela. La aplicación vuelve a guardar la lista sin ese estudiante.

```
=== ELIMINACIÓN DE ESTUDIANTE ===  
  
Ingrese el carné del estudiante (ENTER para cancelar): 116582  
  
Estudiante encontrado:  
  
Carné: 116582  
Nombre: Mario Gonzalez  
Carrera: Ing. Industrial  
Correo: mgonz@correo.url.edu.gt  
  
¿Desea eliminar este estudiante? (S/N): S  
  
Estudiante eliminado correctamente.  
  
Presione ENTER para continuar...  
|
```

Figura 6. Confirmación y eliminación del registro 116582.

Cambiar la contraseña

Seleccione 6. Ingrese la contraseña actual, la nueva y su confirmación. El archivo se descifra con la contraseña actual y se guarda cifrado con la nueva. Utilice la nueva contraseña en los siguientes accesos.

```
=== CAMBIAR CONTRASEÑA ===  
  
Contraseña actual (ENTER para cancelar):  
Nueva contraseña (ENTER para cancelar):  
Confirme la nueva contraseña:  
Contraseña cambiada correctamente.  
Los datos se guardaron cifrados con la nueva contraseña.  
Utilice esa contraseña la próxima vez que abra el programa.  
  
Presione ENTER para continuar...
```

Figura 7. Confirmación del cambio de contraseña.

Si la contraseña actual no permite descifrar el archivo o las nuevas no coinciden, el cambio se rechaza. No hay una función para recuperar una contraseña olvidada. Conserve una copia del archivo y recuerde la contraseña correspondiente a esa copia.

7. Arquitectura del sistema

Las clases pertenecen al espacio de nombres GestionEstudiantes. Proyecto_1.Main prepara la ruta, solicita la contraseña e inicializa la persistencia antes de abrir el menú. Las operaciones reciben un ManejadorXML ya configurado.

Clase	Responsabilidad y métodos principales
Estudiante	Modelo con Carne, Nombres, Apellidos, Carrera y Correo. NombreCompleto se calcula.
MenuPrincipal	Ejecutar controla el menú. Solicita entradas y muestra resultados
EntradaConsola	LeerContrasena y SolicitarNuevaContrasena. Lectura oculta de contraseñas, usada al iniciar y al cambiarla.
Proyecto_1	Clase de arranque. Main prepara la ruta, solicita la contraseña, inicializa ManejadorXML y abre el menú.
OperacionesEstudiante	RegistrarEstudiante, BuscarEstudiantePorCarne, ActualizarEstudiante, EliminarEstudiante y ListarEstudiantes.
Validador	ValidarDatos, ValidarRegistro, ValidarActualizacion, ValidarBusqueda y ValidarEliminacion; auxiliares de comparación.
ManejadorXML	InicializarArchivo, LeerEstudiantes, GuardarEstudiantes y CambiarContrasena. Serialización y persistencia.
CifradorAES	EsArchivoCifrado, Encriptar y Desencriptar. Derivación de clave y autenticación.
ConfiguracionDatos	PrepararUbicacionDatos y PrepararRutaXml. Selección y creación del directorio.

Flujo de una operación

El menú obtiene los datos y llama a OperacionesEstudiante. Esta clase solicita la lectura a ManejadorXML, consulta Validador y aplica el cambio en una lista. ManejadorXML valida y serializa la lista en memoria; CifradorAES la cifra y el contenido se escribe en disco.

Las operaciones de cambio devuelven una tupla con éxito y errores. La búsqueda añade el estudiante encontrado. UltimaLecturaExitosa distingue una lista vacía válida de un error al recuperar los datos.

8. Estructura del XML

El elemento raíz es estudiantes. Cada elemento estudiante contiene cinco campos. El carné identifica el registro; el nombre completo se obtiene uniendo nombres y apellidos, por lo que no necesita una etiqueta propia.

Etiqueta	Tipo en C#	Uso
carne	string	Identificador del estudiante. Uno o varios nombres. Apellidos del estudiante. Carrera académica. Dirección de correo.
nombres	string	
apellidos	string	
carrera	string	
correo	string	

Representación lógica de un registro

```
<?xml version="1.0" encoding="utf-8"?>
<estudiantes>
  <estudiante>
    <carne>116582</carne>
    <nombres>Gerber Mariano</nombres>
    <apellidos>Perez Gomez</apellidos>
    <carrera>Ing. Sistemas</carrera>
    <correo>gmperezgom@correo.url.edu.gt</correo>
  </estudiante>
</estudiantes>
```

Este ejemplo representa el contenido antes de cifrar. En disco, el archivo activo permanece cifrado aunque conserve la extensión .xml. La consulta de sus registros se realiza desde el programa.

Serialización

XmlSerializer transforma los objetos en XML y reconstruye los objetos al leerlo. XmlRoot define la raíz, XmlElement asigna los nombres de los campos, XmlType define el nombre del tipo XML y XmlIgnore excluye NombreCompleto. El constructor público sin parámetros permite crear objetos durante la deserialización.

ContenedorEstudiantes, dentro de ManejadorXML, agrupa List<Estudiante>. Una colección vacía corresponde a <estudiantes />. El serializador representa caracteres como & y < mediante las entidades XML apropiadas.

9. Persistencia y cifrado

La persistencia utiliza XML serializado en memoria y cifrado AES-GCM con una clave de 256 bits. No se guarda la contraseña en el archivo. La clave se deriva de la contraseña con PBKDF2 y SHA-256, utilizando una sal aleatoria.

Parámetro	Valor
Clave AES	32 bytes, equivalentes a 256 bits.
Derivación	PBKDF2 con SHA-256 y 150 000 iteraciones al cifrar.
Sal	16 bytes aleatorios por cifrado.
Nonce	12 bytes aleatorios por cifrado.
Etiqueta de autenticación	16 bytes.
Identificador de formato	MIA-AES-GCM-PBKDF2-1.

El formato guardado concatena el encabezado, el número de iteraciones en cuatro bytes, la sal, el nonce, la etiqueta y el contenido cifrado. El encabezado, las iteraciones y la sal se autentican como datos asociados. Al descifrar se comprueba la etiqueta antes de aceptar el contenido.

Escritura y lectura

Al guardar, se valida la lista, se serializa y se cifra. Los cambios se escriben primero en un temporal cifrado con nombre único, se vacía el búfer y luego se reemplaza el archivo principal. Si el archivo no existe al iniciar, se crea una lista vacía cifrada.

Al leer, se reconoce el encabezado, se descifra y se reconstruyen los estudiantes. Un XML legible existente puede validarse y migrarse automáticamente a formato cifrado durante la inicialización.

Cambio de contraseña y conservación

La nueva contraseña se conserva en la sesión solamente después de guardar con éxito. Cada cambio de estudiantes vuelve a cifrar los datos automáticamente. No es necesario seleccionar una opción para cifrar o descifrar.

Para respaldar, cierre el programa y copie el archivo cifrado. Para restaurar, cierre el programa, conserve el archivo actual y ubique la copia en la ruta activa. La contraseña necesaria será la que protegía esa copia. Evite editar el archivo cifrado con un editor de texto.

10. Validaciones y solución de problemas

El registro comprueba campos obligatorios, formato de correo y coincidencias de carné y correo. La búsqueda y las operaciones de cambio comprueban la existencia del identificador. Los textos se revisan para comprobar que sus caracteres puedan serializarse en XML.

Situación visible	Acción del usuario
Carné inválido	Ingrese el formato solicitado en pantalla y corrija la entrada.
Carné o correo ya registrado	Use otro identificador o revise el estudiante existente.
Datos obligatorios vacíos	Complete nombres, apellidos, carrera y correo.
Estudiante no encontrado	Verifique el carné y la carpeta de datos de la sesión.
Contraseña incorrecta o archivo modificado	Reintente con la contraseña correcta. Si persiste, conserve el archivo y revise un respaldo.
Contraseñas nuevas diferentes	Escriba de nuevo la contraseña y su confirmación.
No se puede escribir en la carpeta	Revise permisos y configure una ruta accesible.

Ruta y archivo activo

Si no aparecen los estudiantes esperados, revise primero la ruta que muestra la aplicación. Ejecutar el script y ejecutar directamente pueden seleccionar carpetas distintas. La extensión .xml no significa que el archivo cifrado se pueda abrir como XML legible.

Mantenimiento

Utilice una copia independiente de los datos para pruebas de archivos dañados. Conserve el original antes de reemplazarlo. La aplicación carga y vuelve a guardar la lista completa; conviene trabajar con una sola instancia para evitar que dos sesiones sobrescriban cambios.

El documento de evidencias diferencia las validaciones visibles en el menú de las pruebas adicionales de persistencia, cifrado e integridad. Una captura de un caso correcto no certifica todos los posibles errores de entrada.

11. Control de versiones

Git conserva la evolución del código mediante commits. Las ramas permiten trabajar en módulos y después integrar los cambios. El repositorio de trabajo del equipo es Proyecto_MIA_GRUPAL_TILINES, en la cuenta savilleda.

Repositorio de trabajo: https://github.com/savilleda/Proyecto_MIA_GRUPAL_TILINES **Historial local del proyecto**

Commit	Descripción registrada
0e344c5	fix: arregla errores de sintaxis en el código
e1f32bc	Cambios en MenuPrincipal y Validador y limpieza de consola.
b4012de	Nuevo flujo e integración de cifrado, configuración y persistencia.
1bfa40d	Preparación de archivo ejecutable.

El historial muestra cambios de integración y correcciones. Para demostrar la participación individual deben mostrarse también los autores y los aportes de cada integrante.

Evidencia del repositorio

Evidencia de los archivos del repositorio:

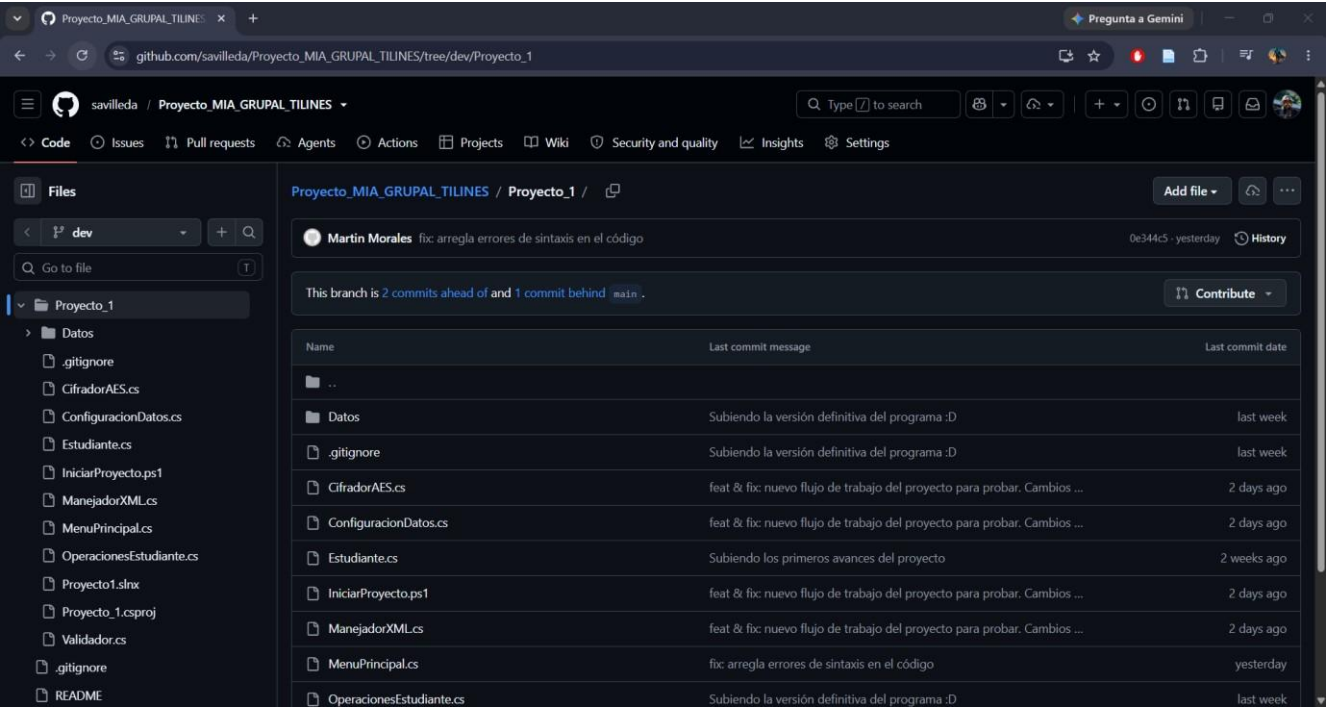


Figura 8. Archivos principales.

Diferentes ramas trabajadas:

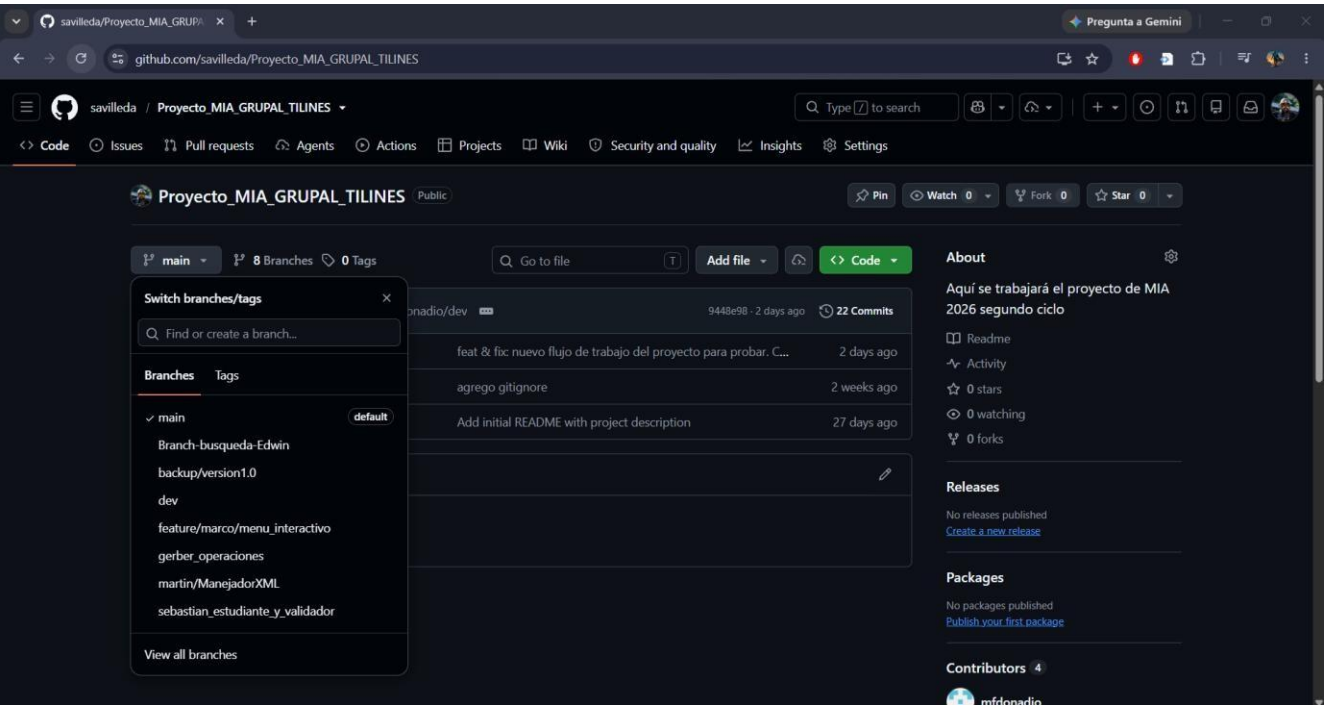


Figura 9. Muestra de ramas

Commits en main:

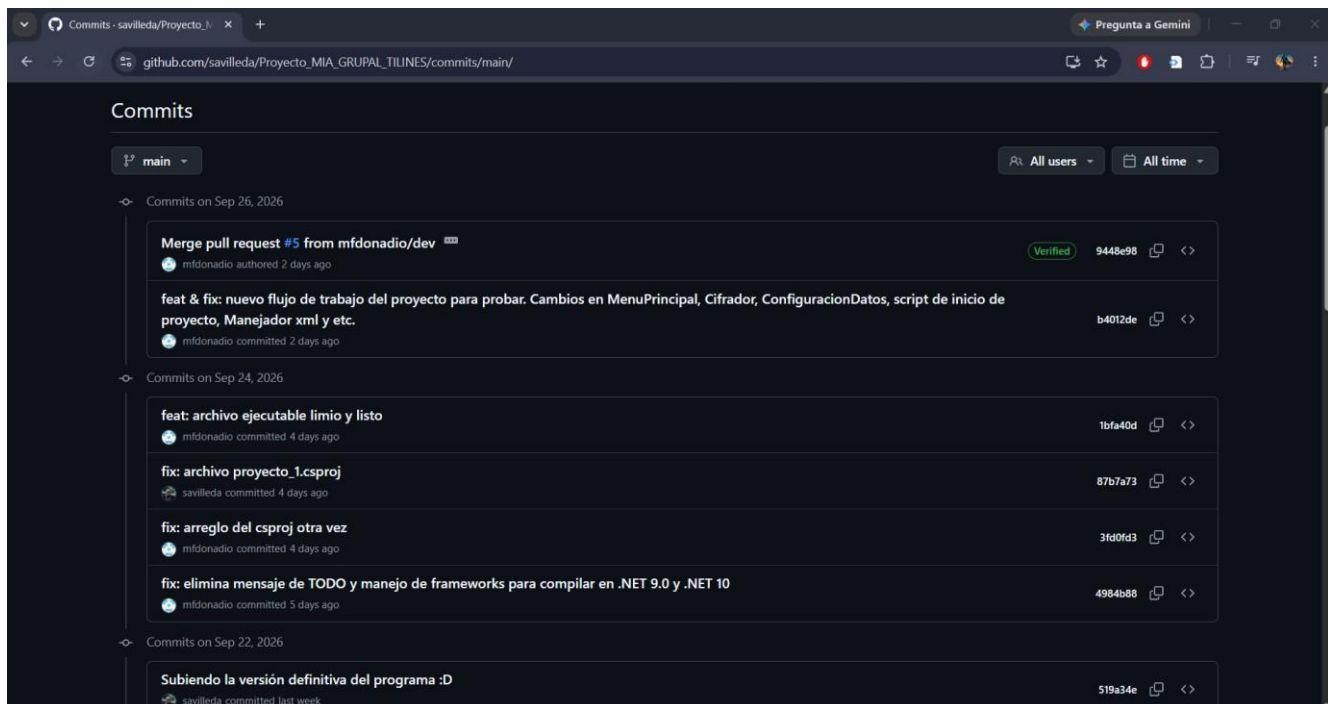


Figura 10. Muestra de commits en el main.

Commits en dev:

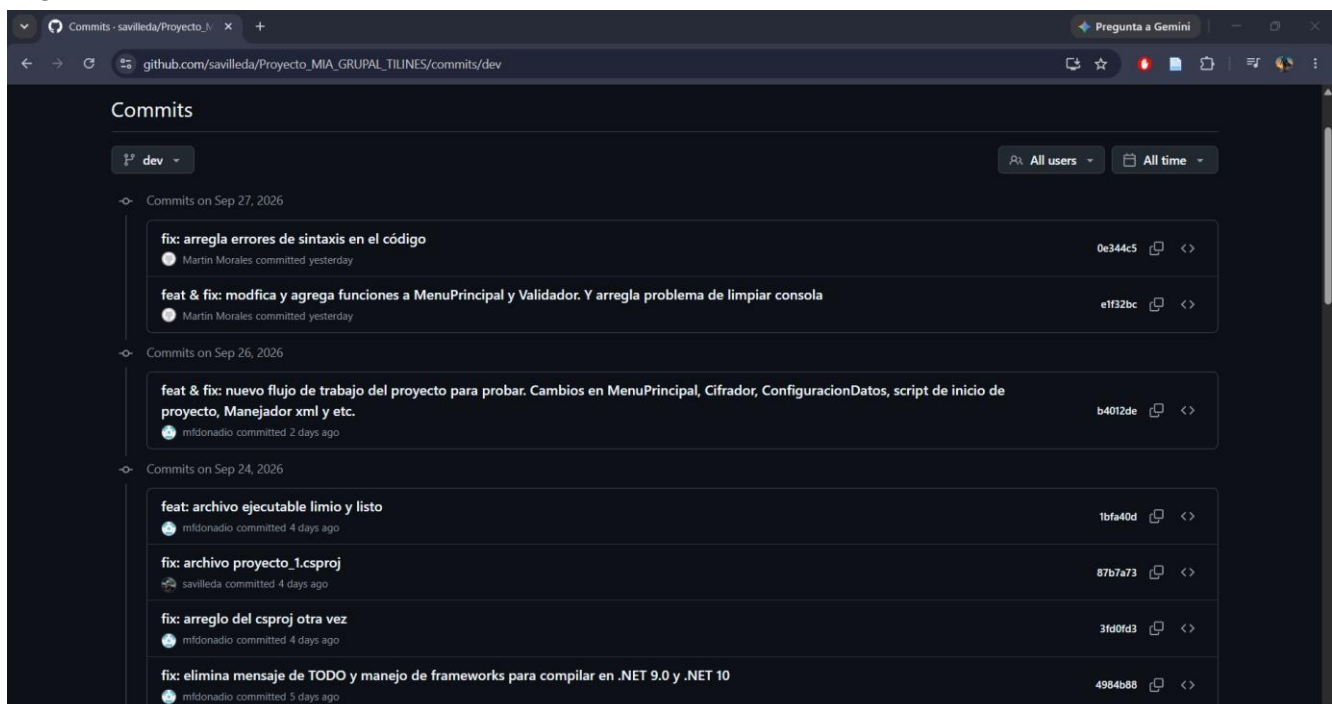


Figura 11. Muestra de commits en el dev.

El docente autorizó GitHub como repositorio remoto para esta entrega. Enlace: https://github.com/savilleda/Proyecto_MIA_GRUPAL_TILINES

Los archivos bin y obj corresponden a resultados de compilación y no sustituyen al código fuente. La entrega debe conservar el proyecto completo, sus archivos utilizados y los dos documentos PDF.

12. Conclusiones y referencias

Conclusiones

El sistema reúne las operaciones básicas de gestión de estudiantes en una consola y utiliza el carné para localizar registros. Las capturas muestran creación, consulta, actualización y eliminación, además de respuestas ante entradas inválidas.

La estructura XML mantiene relacionados los campos de cada estudiante. La serialización permite convertir esa estructura en objetos de C# para aplicar cambios y volver a guardarla.

El cifrado automático protege el contenido almacenado y vincula el acceso a una contraseña. Su uso requiere conservar tanto el archivo como la contraseña correcta y distinguir la carpeta activa de otras copias de datos.

La separación de clases facilita explicar el recorrido de la información desde el menú hasta el archivo. La búsqueda actual recorre la lista reconstruida; para volúmenes mayores, una mejora posible sería incorporar un índice y revisar la coordinación de escrituras.

Material de referencia

1. Proyecto de Aplicación No. 1 - 2026S2. Sistema de Gestión de Información basado en XML. Manejo e Implementación de Archivos. Enunciado y entregables, páginas 1 a 5.
2. Código fuente del proyecto: Estudiante.cs, Validador.cs, OperacionesEstudiante.cs, ManejadorXML.cs, CifradorAES.cs, ConfiguracionDatos.cs MenuPrincipal.cs, Proyecto_1.cs y EntradaConsola.cs. Configuración: Proyecto_1.csproj e IniciarProyecto.ps1.
3. Capturas.docx. Evidencias de ejecución aportadas por el equipo, capturas 1 a 33.
4. Flujo de Trabajo y Organización - Proyecto 1. Documento de planificación de responsabilidades, fases y ramas.